

pop: POP-LEGAL-06

título: Checagem preventiva de glifo antes de gerar/regenerar qualquer PDF

area: Legal — ferramental de produção (md_to_pdf.py, gerar_prancha_legal.py)

autor: Kelsen (Gestor Legal)

criado: 2026-08-07

origem: Autoauditoria de rotina (sem pendência aberta na fila) — padrão de erro recorrente identificado nos próprios registros de estado (Kelsen e Hely), não pedido de cliente nem de Wallenberg.

status: oficial — POP de Kelsen, autonomia de Gestor sobre POP próprio (22/07/2026)

princípios: 8 (Rastreabilidade), 9 (Padronização), 18 (Ética e conformidade — documento que chega a cliente/prefeitura não pode ter texto silenciosamente corrompido)

POP-LEGAL-06 — Checagem preventiva de glifo antes de gerar/regenerar PDF

1. Objetivo e por que existe

`_ferramentas/md_to_pdf.py` (xhtml2pdf/Helvetica) e `_ferramentas/gerar_prancha_legal.py` (latin-1) **não lançam erro** quando o texto de origem contém caractere fora do conjunto que a fonte/encoding suporta — o glifo é **descartado em silêncio**. `result.err` do `md_to_pdf.py` fica `False` mesmo quando isso acontece. Este bug se repetiu **três vezes** entre 21/07 e 28/07/2026 (seta unicode "seta-para-direita" em POP-GESTOR-LEGAL-01.md, depois em POP-LEGAL-02.md, depois em SKILL.md; emoji de alerta em SKILL.md), sempre descoberto **depois** do fato, por rasterização manual do PDF já gerado — nunca antes, por checagem preventiva. Este POP fecha essa lacuna de processo. (Nota irônica, achada ao gerar o PDF deste próprio POP em 07/08/2026: os glifos literais citados neste parágrafo, no original em .md, sumiram na primeira versão do PDF — a prova viva do bug que o documento descreve. Corrigido para texto descritivo aqui; os glifos originais só existem de fato no .md fonte.)

2. Regra de ouro deste POP

Antes de rodar `md_to_pdf.py` ou `gerar_prancha_legal.py` sobre qualquer arquivo `.md/.json` editado (por Hely ou por Kelsen), rodar uma varredura de caracteres fora do intervalo seguro e resolver antes de gerar — nunca depois.

3. Procedimento

1. **Antes de gerar o PDF**, rodar (Bash, ambiente com Python/grep): `grep -nP '[^\x00-\x7F\xC0-\xFFÀ-■]' <arquivo.md ou .json>` ou, mais simples e direto ao problema já visto: buscar especificamente por setas, emojis e símbolos matemáticos comuns (lista viva — ver seção 6): `grep -n '→|←|■|✓|✗|■|■|≥|≤|≈|±' <arquivo>` (10/08/2026: adicionados `≥|≤|≈|±` à lista — achado real de Hely ao rasterizar `_indice_fontes.pdf` inteiro, 8 ocorrências pré-existent de `≥|≈` de 27/07 e 30/07, escritas antes deste POP existir, nunca tinham passado por rasterização de documento inteiro. Fecha a lacuna que a seção 6 já previa. Ver `pendencias.json`, id `b15-glifo-symbol-indice-fontes`.) (Nota, 07/08/2026: este comando com os

glifos literais é o que Hely deve copiar e rodar de fato — funciona normalmente em Bash/UTF-8. Só a **geração de PDF** deste POP a partir do `.md` descarta esses glifos em silêncio, mesma classe de bug que o POP descreve; por isso no PDF esta linha pode aparecer com lacunas — o comando real e íntegro está sempre no `.md` fonte, nunca no PDF.)

2. **Se encontrar ocorrência:** decidir por substituição ASCII equivalente quando o glifo é só marcador (seta-para-direita vira `->`, alerta vira `[ATENÇÃO]`), preservando o sentido — não é permitido gerar o PDF com a ocorrência intacta e "torcer" para que renderize.

3. **Rodar o script de geração.**

4. **Confirmar por rasterização** (não só "rodou sem erro") — abrir ao menos a página onde estava o glifo suspeito e ler visualmente. `result.err == False` não é confirmação de conteúdo correto (ver seção 1).

5. Se o arquivo de origem tem acentuação **legítima** em português (á, ç, ã, etc.) — isso é diferente do problema deste POP. `latin-1` cobre acentuação portuguesa normalmente (ver diagnóstico de 21-23/07, item B3 em `pendencias.json`: a causa da prancha sem acento era ausência de acentuação no **dado de origem**, não falha do encoding). Este POP trata só de glifos **fora** do latin-1/Helvetica-padrão (setas unicode, emoji, símbolos especiais) — não confundir os dois diagnósticos.

4. Escopo

Aplica-se a qualquer arquivo que alimente `md_to_pdf.py` ou `gerar_prancha_legal.py`: POPs, `_indice_fontes.md`, `SKILL.md`, `caso_prancha.json`, pareceres/adendos gerados em PDF. Hely executa o passo 1-4 como parte padrão de qualquer tarefa que termine em "regenerar PDF" — não é passo opcional nem depende de pedido explícito de Kelsen a cada vez.

5. Confiança

Alta — os três incidentes-fonte estão documentados e auditados (rasterização) em `pendencias.json` (`b1-regerar-pdfs`, `b2-pop-legal-02-quarentena`) e em `_estado_kelsen.md`, seção 3 ("A correção que o Hely fez num arquivo não garante que a mesma classe de bug não sobreviva em outro arquivo mudado na mesma rodada"). O procedimento aqui é a formalização preventiva dessa lição, que até hoje só existia como aprendizado reativo, não como passo de processo.

6. Lacunas conhecidas

- O regex do passo 1 é abrangente mas não exaustivo — símbolos unicode fora dos exemplos já vistos podem escapar. Se aparecer um novo incidente com glifo diferente dos já listados, adicionar o padrão específico à lista do passo 1 (não reescrever o POP do zero).
- **10/08/2026 — 4º incidente fechado desta forma:** $\geq/\approx$ (e por precaução $\leq/\pm$, mesma família "Symbol") achados por Hely ao rasterizar `_indice_fontes.pdf` inteiro (8 ocorrências pré-existent de 27/07 e 30/07, escritas antes deste POP existir). Adicionados ao exemplo de grep do passo 1. Correção do conteúdo já arquivado em `pendencias.json`, id `b15-glifo-symbol-indice-fontes`.

- Não cobre PDFs gerados por outra ferramenta que não essas duas (nenhuma outra identificada em uso hoje).
- Rasterização de página isolada (o padrão usado nos incidentes 1-3) só pega o glifo se o auditor souber onde olhar; rasterizar o **documento inteiro** (como Hely fez em 10/08, fora do escopo do pedido original) é o que efetivamente achou o 4º incidente, pré-existente e não relacionado à edição do dia. Considerar, em documento extenso (`_indice_fontes.md`, `SKILL.md`), tratar rasterização integral como parte do passo 4 quando o arquivo for regenerado por qualquer motivo — não só a página tocada na rodada.